

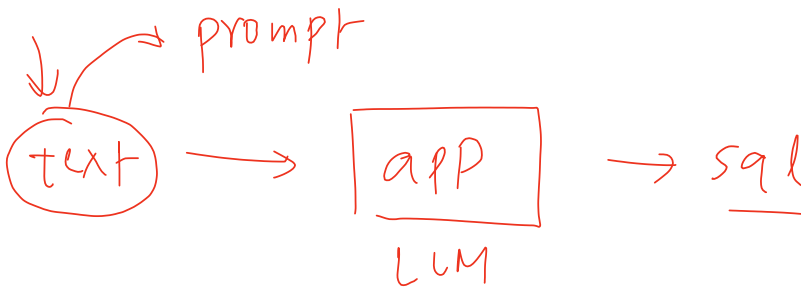
Requirements

Wednesday, 29 July 2026 5:36 PM

1. Task → Text2SQL
2. Cost → Retention → 3 lacs
3. Latency → 2-3 sec
4. Context
5. Deployment
6. Correctness

opensource
public api

Text2 SQL system
Context LLM per token charge



SYSTEM:

You are a text-to-SQL generator. Given a database schema return a single SQL query that answers it. Use SQLite the SQL query.

USER:

Schema:

```
CREATE TABLE "matches" (
  "id" INTEGER, "season" TEXT, "city" TEXT, "date" TIMESTAMPTZ,
  "match_type" TEXT, "player_of_match" TEXT, "venue" TEXT,
  "team1" TEXT, "team2" TEXT, "toss_winner" TEXT, "winner" TEXT,
  "result" TEXT, "result_margin" REAL, "target_overs" REAL,
  "super_over" TEXT, "method" TEXT, "umpire1" TEXT, "umpire2" TEXT
);

CREATE TABLE "deliveries" (
  "match_id" INTEGER, "inning" INTEGER, "batting_team" TEXT,
  "bowling_team" TEXT, "over" INTEGER, "ball" INTEGER,
  "bowler" TEXT, "non_striker" TEXT, "batsman_runs" INTEGER,
  "extra_runs" INTEGER, "total_runs" INTEGER, "extra_is_wicket" INTEGER,
  "player_dismissed" TEXT, "disfielder" TEXT
);
```

Question: Which bowler has the best economy rate at least 500 legal balls? Economy is runs conceded per over. Exclude wides and no-balls from the ball count. Give economy.

SQL:

```
SELECT bowler,
  6.0 * SUM(total_runs) /
  SUM(CASE WHEN extras_type IS NULL
    OR extras_type NOT IN ('wides', 'noballs')
    THEN 1 ELSE 0 END) AS economy
FROM deliveries
GROUP BY bowler
HAVING SUM(CASE WHEN extras_type IS NULL
  OR extras_type NOT IN ('wides', 'noballs')
  THEN 1 ELSE 0 END) >= 500
ORDER BY economy ASC
LIMIT 1
```

4:1

→ 400 tokens
→ 100 tokens

34

400 tokens → 100

on avg → daily → question.

50K amount upto INR in

3.1k - 1000000

101

1 query $\rightarrow$ 400 input / 100 output 50

$$50000 \times 30 \times (95)$$

per query wst

$$\frac{10}{1000000} \times 400 + \frac{50}{1000000} \times$$

$$\frac{9}{1000} = 0.$$

$$50k \rightarrow 0.009 \times 50000$$

$$= \underline{250000} =$$

(3L)

$$250 \times 30 = 7500$$

$$450000 = 45$$

$$\text{month } 450 \times 30 = [13]$$

[prompt
caching]

$$(12.82) \text{ L/m } [13500 \times 9]$$

qurst $\rightarrow 10/12$

5m cache (2)

1 qurst $\rightarrow$ cache 7
 $\uparrow$

3m

(50)

1000 qurst m

K V-char cache

- 1) Req $\nearrow$ Text + 2 SQL leaders
2) Leaderboard $\rightarrow$ 5-10 candidates

$\downarrow$
coding
 $\rightarrow$ leaderboard

short list

$$\frac{4 \times 10 + 1 \times 50}{5} = \frac{90}{5} = 18$$

400 + 100
 $\left[\begin{array}{l} 12L \\ 8L \end{array} \right]$

$$500 \times \frac{(18)}{100000} \times 500 \times 3.1 \times 95$$

total cost

output $\rightarrow$ 3L $\rightarrow$ 100 reqs
SQL query

now malize

50 - min normal

50 - min

0.2

→ $(0.9) \times \text{normal rating score} + (0.1) \times \text{normal latency score}$

best

(5L) / 3L

1) 4P

ST - reg.

St. 5-10

SRS - $Cu_{2}S$

→ ① dataset → Data
② golden dataset analysis

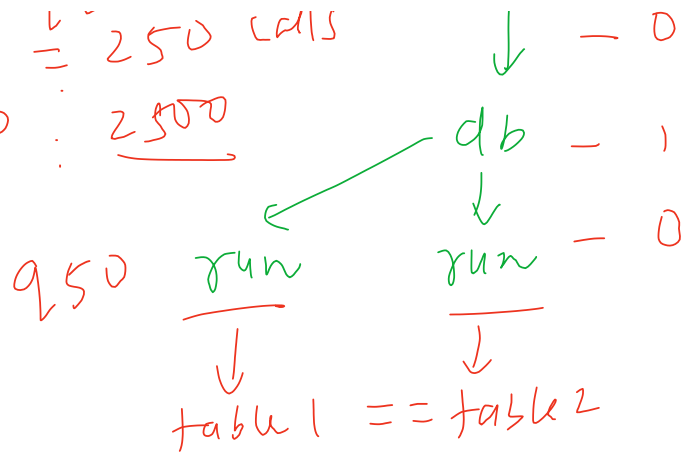
30-50 → questions 10 easy
→ ① ————, SQL query 20 med
 ↑
 ② ————, SQL query 20 hard

50 20 5.6 - terra $\rightarrow$ 95%

(3) $a \perp$ → kein $\downarrow$ —
—
 $a \vdash$

$\searrow [gan_sql]$

$$5 \times 50 = 250 \text{ calls}$$
$$\begin{array}{r} 500 \\ \hline 2500 \end{array}$$



2008-2024

2020-2024

20 quarters

- ✓ 1. The dataset
- ✓ 2. ~~Filter and load the Database~~
- ✓ 3. ~~Extract the schema~~
- ✓ 4. Build the golden dataset
- ✓ 5. Validate the golden dataset
- ✓ 6. Store the golden dataset in csv format
- ✓ 7. Check the flow
- ✓ 8. Pick candidate models
- ✓ 9. Build the evaluator
- ✓ 10. Build the orchestrator
11. Run all the models
12. Make the decision

⑤ $\left(\begin{array}{c} \text{open} \\ \text{high} \\ \text{low} \\ \text{close} \end{array} \right)$

⑨ $\rightarrow$ LLM
 $\downarrow$
SQL

2.9999 2.99

table 1

table 2

2.0 2

Order matters

row

same same

values normalized

table

sort X

$\rightarrow \left[\begin{array}{ccc} \text{(row 1)} & \text{(row 2)} & \text{(row 3)} \end{array} \right]$

$\rightarrow \text{(row 1)} \quad \text{(row 2)} \quad \text{(row 3)}$

sort

$\downarrow$

